HOTEL BOOKING MANAGEMENT SYSTEM

Java • JDBC • MySQL • Maven • IntelliJ IDEA

PROJECT REPORT

A console-based application to create, view, search and cancel hotel bookings, with persistent data stored in a MySQL database.



1. Project Overview

The Hotel Booking Management System is a Java application designed to manage hotel booking records through a menu-driven interface. JDBC connects the Java application to MySQL, while the DAO layer contains SQL operations. Maven manages the MySQL Connector/J dependency.

Core functions: add a booking, view bookings, search for a booking, cancel a booking, and connect to the hotel_booking_db database.

2. Technologies Used

Technology	Purpose
Java 17	Application logic and object-oriented model
JDBC	Connects Java code to MySQL
MySQL	Persistent booking storage
Maven	Dependency and project configuration
IntelliJ IDEA	Development environment

3. Problem Statement, Aim and Objectives

3.1 Problem Statement

Hotel booking information needs to be recorded and retrieved accurately. When booking details are handled manually or kept in scattered records, searching for a reservation, reviewing its details, and removing a cancelled booking can take extra time. A small computerized system can organize these operations and keep the information available after the program closes.

The Hotel Booking Management System addresses this basic requirement through a console-based Java application connected to a MySQL database. The report describes the functions evidenced by the supplied project: adding a booking, viewing saved bookings, searching for a booking, and cancelling a booking. It is a foundational record-management application rather than a complete commercial hotel platform.

3.2 Aim of the Project

The main aim is to develop a simple, menu-driven application that demonstrates how a Java program can manage hotel booking records and store them persistently in a relational database using JDBC.

3.3 Objectives

- Provide a clear menu through which a user can select a booking operation.
- Accept booking information and store it in MySQL.
- Display stored booking records when requested.
- Search for a particular booking using its identifying information, such as a booking ID where supported by the implementation.
- Allow a selected booking to be cancelled and remove its record as described in the supplied report.
- Separate the booking data model, database operations, and connection setup into distinct classes.
- Use Maven to manage the project configuration and database driver dependency.

3.4 Scope of the System

The current scope is limited to core booking-record operations from a console interface. The report does not establish that the application currently handles room inventory, payment processing, customer login, email notifications, online booking, or simultaneous users. These are therefore treated as possible future improvements, not as existing features.

4. System Design and Architecture

4.1 Overview of the Design

The application follows a simple layered arrangement. The user interacts with the console menu, the application code handles the selected operation, the Booking class represents booking data, and BookingDAO performs database-related work. DBConnection centralizes the process of opening a connection to MySQL. This separation makes the program easier to understand because each class has a focused responsibility.

4.2 Main Components

Component	Responsibility
Main.java	Displays the welcome/menu interface, receives the user's choice, and directs the program flow to the selected operation.
Booking.java	Acts as the booking model: a Java representation of booking information passed between program components.
BookingDAO.java	Contains data-access operations and SQL interactions used to create, retrieve, search, or cancel booking records.
DBConnection.java	Keeps the database connection setup in one place, including the connection details needed by JDBC.
MySQL database	Stores booking records persistently so that saved information can be retrieved in later program runs.
Maven / pom.xml	Defines project configuration and manages external libraries such as the MySQL JDBC driver.

4.3 Typical Flow of an Operation

A typical interaction begins when the user starts the program and sees the menu. The user chooses an operation and supplies the requested input. The application then calls the relevant database-access method. JDBC sends the SQL operation to MySQL, and the result is returned to the Java program. Finally, the program displays a result or confirmation in the console. The exact prompts and messages depend on the implementation shown in the supplied screenshots.

4.4 Benefits of Modular Design

Separating the model, menu logic, and database access reduces the need to place all code in one large class. If database logic changes, it can generally be handled in the DAO layer. If the user interface is later replaced with a graphical or web interface, the booking model and much of the database logic may be reusable with suitable modifications.

5. Core Concepts and Technologies

5.1 Java and Object-Oriented Programming

Java is the main programming language used for the application. It supports classes and objects, which help represent real-world entities in program form. In this project, Booking is the model class for a booking record. Keeping related booking data together in a class makes the information easier to pass to and from methods. The Main class coordinates user interaction, while other classes handle specialized tasks.

5.2 JDBC (Java Database Connectivity)

JDBC is the standard Java API for communicating with relational databases. A Java application uses a database driver and a connection to send SQL statements to a database. The result of a query can be read and converted into Java values for display or further processing. In this project, JDBC forms the bridge between Java code and MySQL.

A typical JDBC interaction includes obtaining a connection, preparing or executing an SQL statement, processing the result if one is returned, and closing database resources. Using prepared statements for user-supplied values is a recommended practice because it helps separate SQL structure from input data and reduces SQL-injection risk. The exact JDBC methods used should be confirmed against the submitted source code.

5.3 MySQL and Relational Data

MySQL is a relational database management system. It stores information in tables made up of rows and columns. A booking table can represent one booking per row, with columns for the fields required by the application. A unique booking identifier is commonly used to distinguish one record from another. The supplied report identifies the database as hotel_booking_db; the exact table and column structure should follow the SQL schema used with the project.

Persistent storage is important because data held only in program variables disappears when the application ends. When records are saved to MySQL, they can be queried again during a later run, provided the database and connection configuration remain available.

5.4 Maven and IntelliJ IDEA

Maven manages project dependencies and build configuration through the pom.xml file. For a JDBC project, it can declare the MySQL Connector/J library so the driver is available to the application. IntelliJ IDEA is the development environment used to organize source files, configure the project, run the program, and inspect errors. These tools support development; the booking data itself is stored in MySQL rather than in Maven or the IDE.

6. Database Operations and Data Handling

6.1 Create, Read, Search and Cancel

The application's main features can be understood as common database operations. Adding a booking creates a new record. Viewing bookings retrieves records for display. Searching narrows the result to a booking that matches the supplied identifier or search criteria. Cancelling a booking removes the selected record according to the current project description.

Operation	Database idea	Expected behaviour
Add booking	INSERT	The new booking is stored and can be retrieved later.
View bookings	SELECT	Saved booking records are displayed in the console.
Search booking	SELECT with a condition	The matching record is displayed, or the program reports that no match was found.
Cancel booking	DELETE or status update	The selected booking is cancelled using the approach implemented by the project.

6.2 Why Input Validation Matters

Users may enter blank values, letters where a numeric identifier is expected, or an ID that does not exist. Validation helps prevent incorrect data from reaching database operations. A robust version should check required fields, ensure values use the expected format, and give understandable messages when input is invalid. Database errors should also be handled gracefully so that one failed operation does not unexpectedly terminate the application.

6.3 Data Integrity and Safe Updates

Data integrity means keeping stored records accurate and consistent. A primary key or unique identifier can prevent records from being confused with one another. Database constraints can enforce required values, while transactions can help ensure that related changes are completed together. For a small academic project, these concepts are useful design considerations even if the current implementation uses only basic operations.

6.4 Security Considerations

Database credentials should not be published in screenshots, source repositories, or public reports. A safer implementation can read credentials from environment variables or a configuration file excluded from version control. Queries should use parameterized statements, and database accounts should have only the permissions required by the application. The supplied report obscures the password in its connection screenshot; the same care should be maintained in any shared copy.

7. Testing, Limitations and Future Enhancements

7.1 Testing Approach

Testing checks whether each feature behaves as expected for both normal and unusual inputs. The existing report includes a checklist for connecting to the database, adding, viewing, searching, and cancelling bookings, including handling an invalid booking ID. A systematic test run should record the input, expected result, actual result, and pass/fail status.

Test scenario	Expected result
Valid database credentials	The application establishes a connection to MySQL.
Add valid booking details	The record is saved and appears when bookings are viewed.
Search for an existing booking	The correct booking details are shown.
Search for an unknown ID	A clear no-record-found message is shown without crashing.
Cancel an existing booking	The application confirms the action and the record is no longer returned, if deletion is the implemented approach.
Enter invalid or incomplete input	The application gives a useful message and continues safely where possible.

7.2 Current Limitations

Based on the supplied report, the application currently focuses on console-based booking-record management. The report does not demonstrate a graphical interface, online access, role-based authentication, automated room-availability checks, payment processing, billing, or notification services. These should not be described as completed features unless they are added and tested.

7.3 Future Enhancements

Several improvements could make the system more useful. Room availability checking could prevent overlapping reservations. Date validation could ensure check-out is after check-in. A graphical or web interface could make the system easier to use. Authentication and role permissions could separate receptionist and administrator actions. Billing and payment records could support financial tracking, while email or SMS notifications could inform customers about booking status. Automated tests, logging, and environment-based configuration would also improve reliability and maintainability.

7.4 Learning Outcomes

This project provides practical experience with Java classes, a menu-driven program, database connectivity, SQL-based data access, and dependency management. It also illustrates why modular design, input validation, testing, and credential security matter in applications that store user-related records.

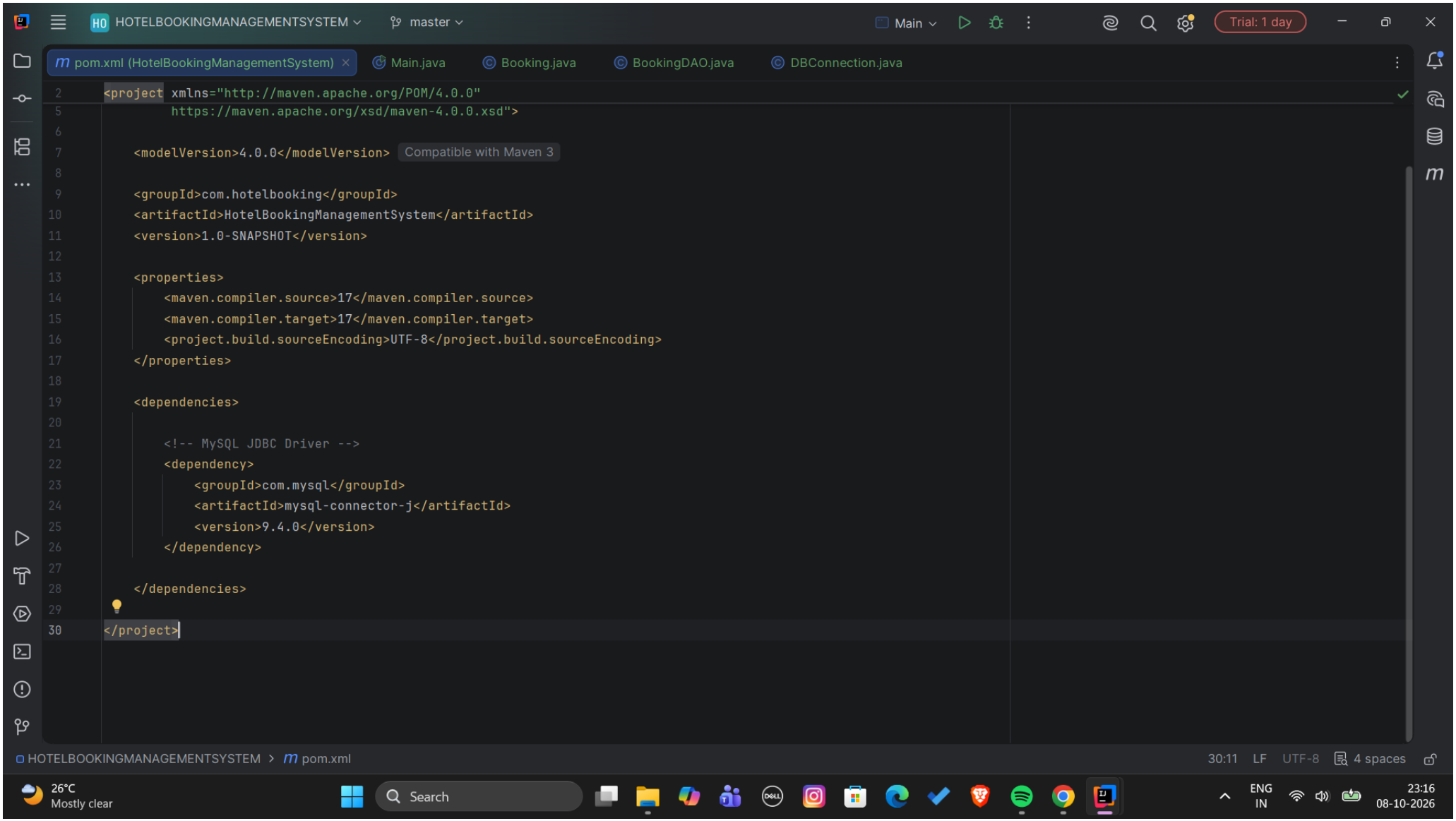
7.5 Summary

The Hotel Booking Management System is a useful introductory example of connecting a Java application to a relational database. Its core operations demonstrate how booking records can be created, retrieved, searched, and cancelled. The architecture also provides a starting point for adding stronger validation, security, and hotel-management features in later versions.

8. Project Code Screenshots

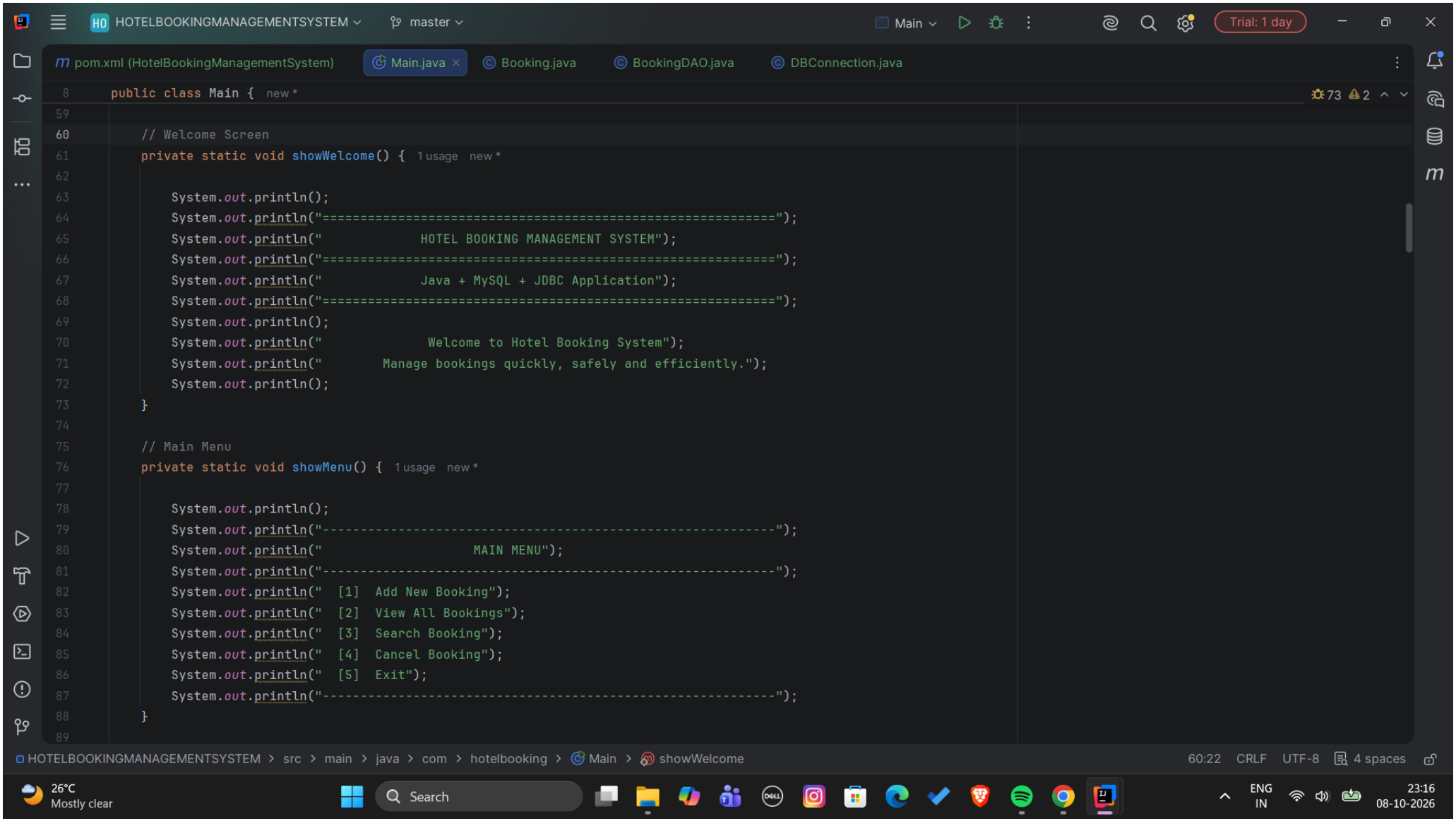
The following pages contain the screenshots supplied by the project author. The database password value is hidden in the DBConnection screenshot.

Maven Configuration — pom.xml



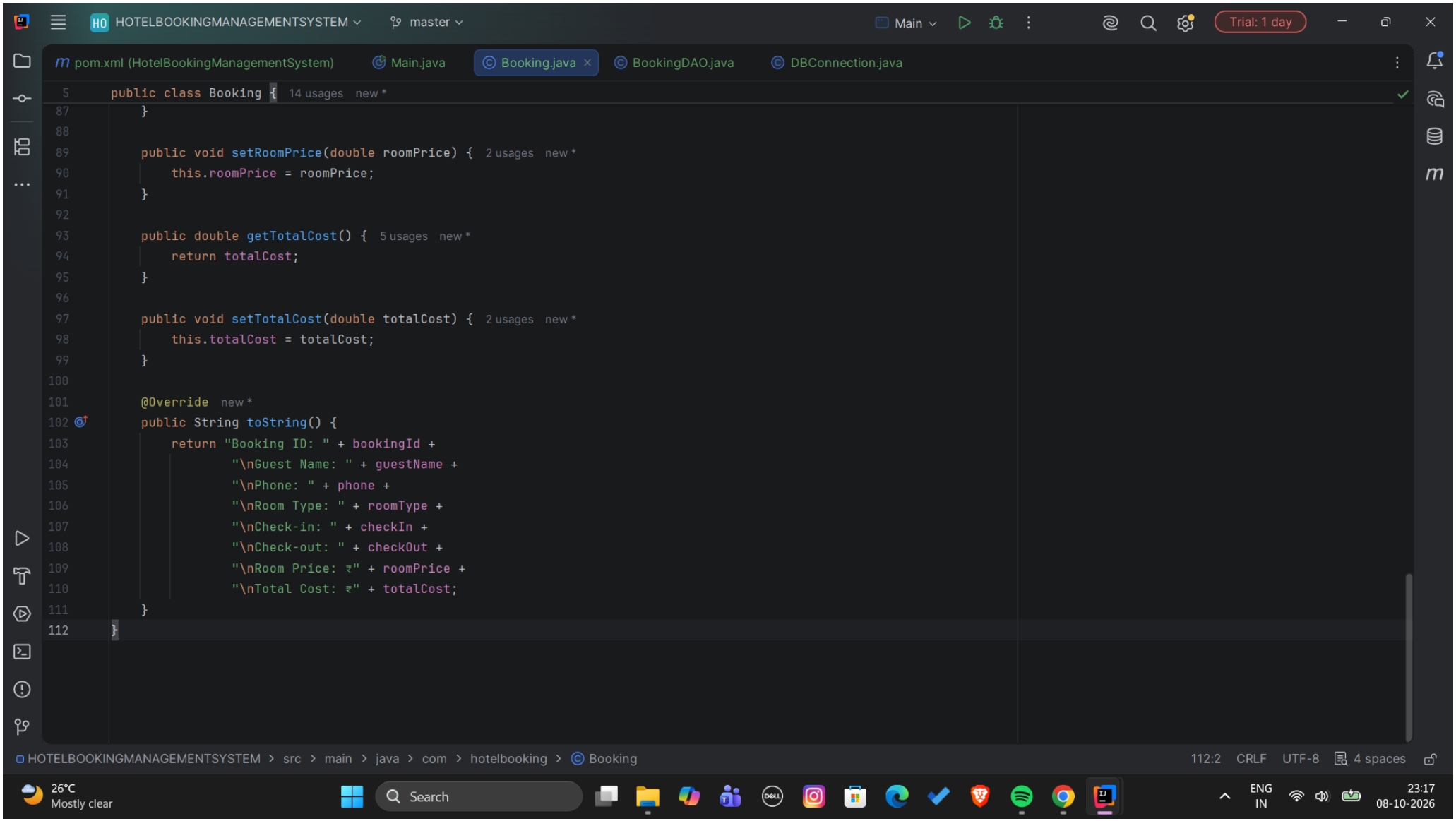
Submitted code screenshot

Main.java — Welcome Screen and Main Menu



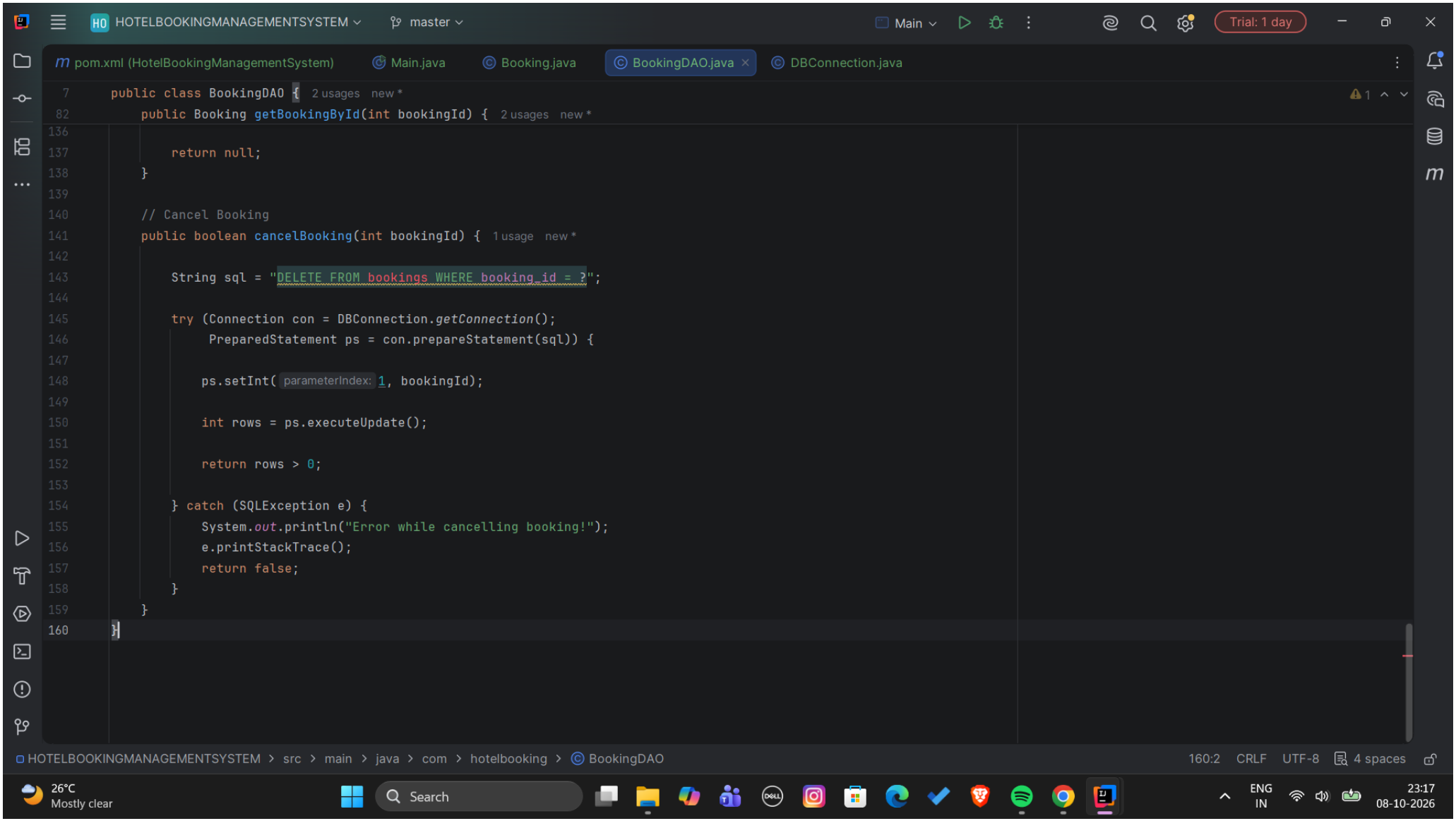
Submitted code screenshot

Booking.java — Booking Model



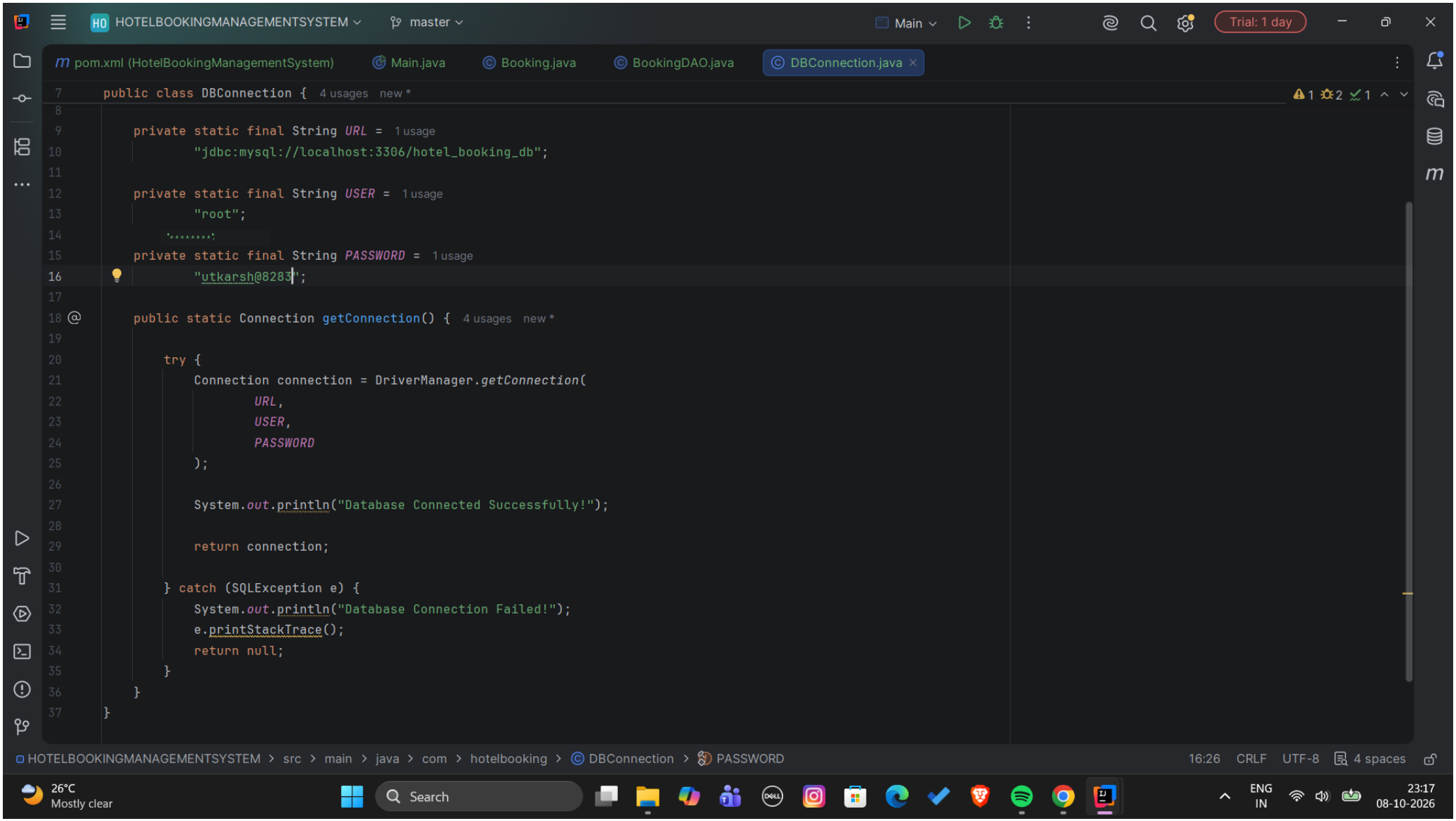
Submitted code screenshot

BookingDAO.java — Database Operations



Submitted code screenshot

DBConnection.java — Database Connection

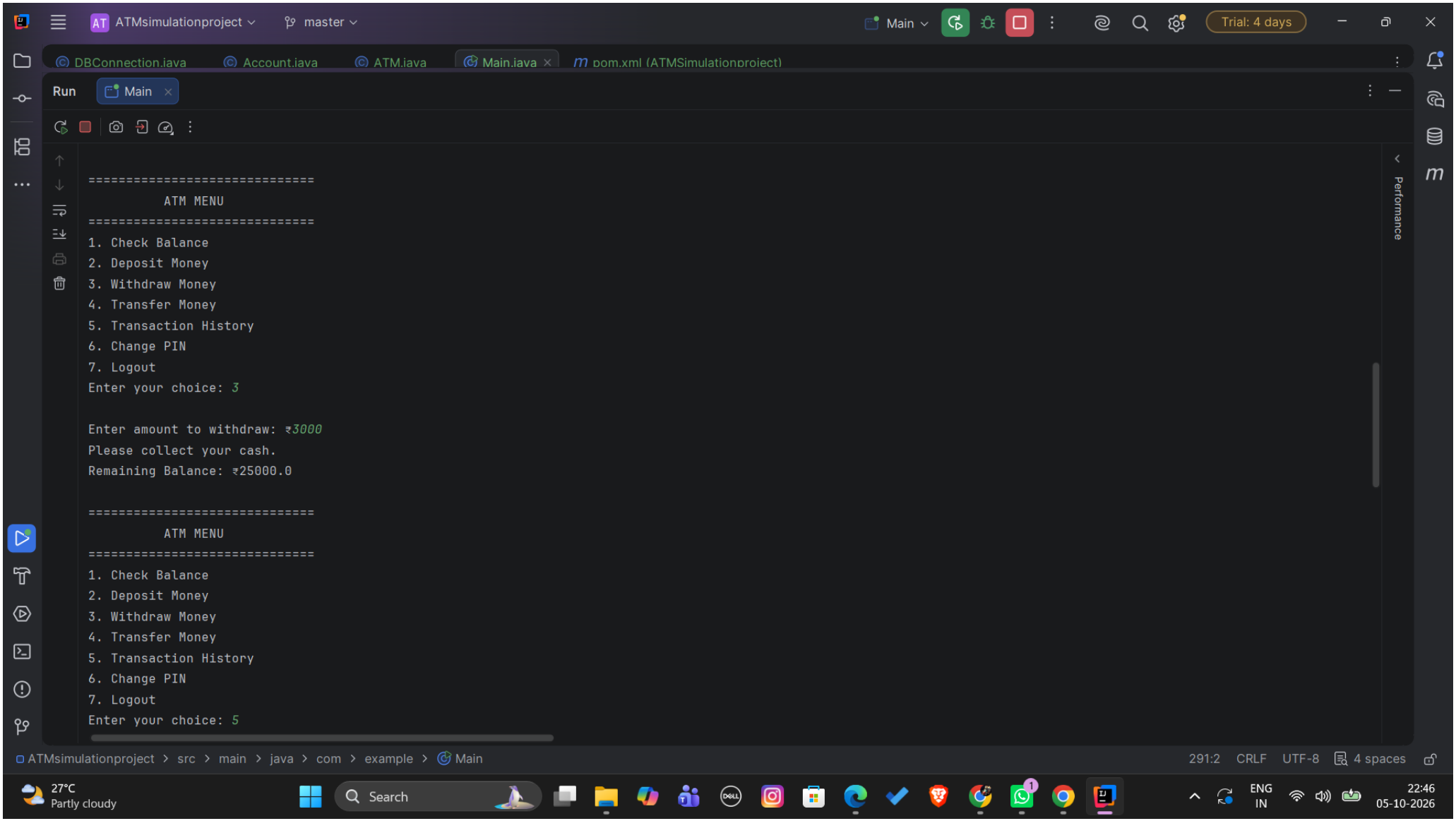


Submitted code screenshot

9. Application Output Screenshots

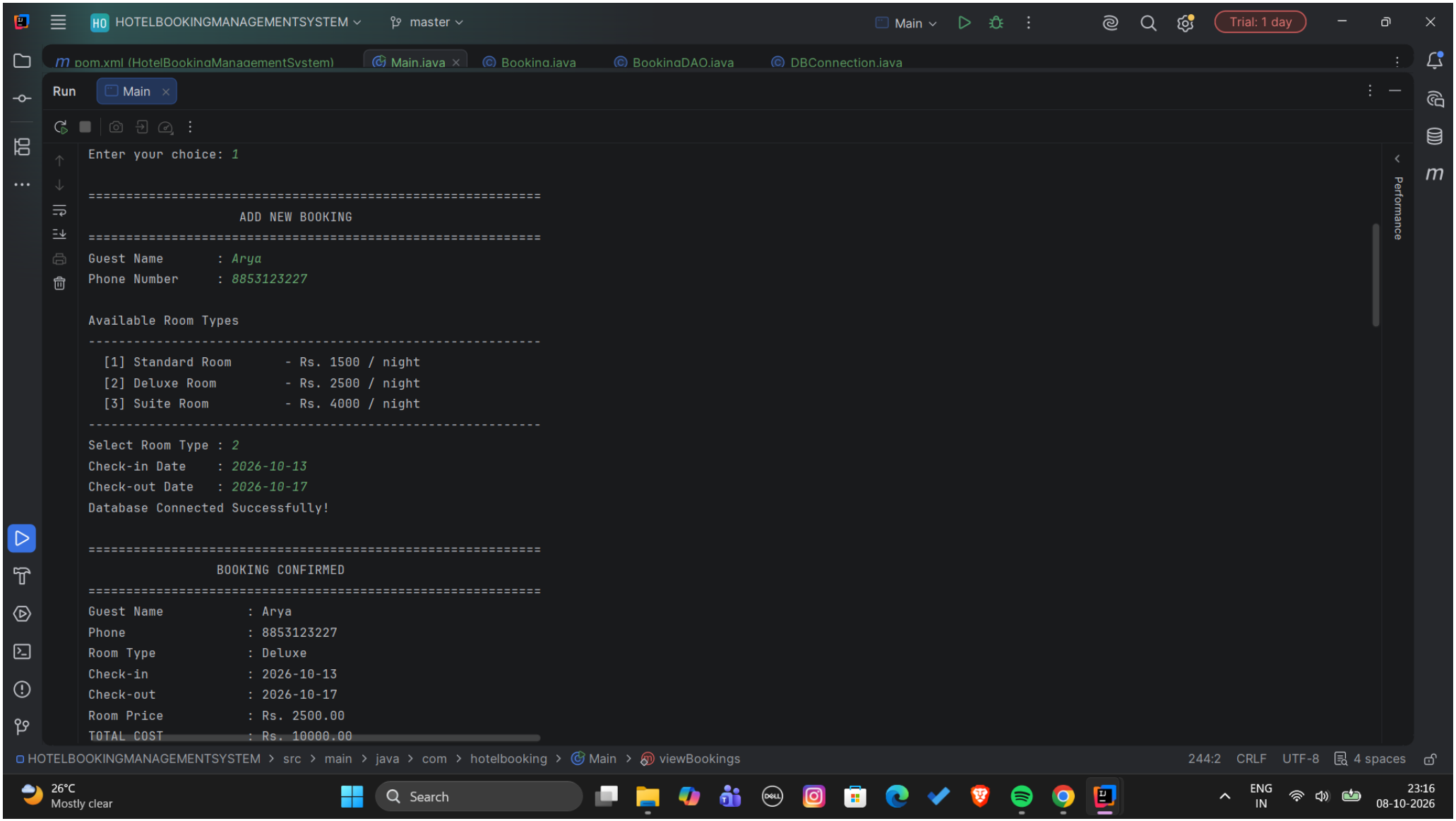
The supplied console screenshots are included as evidence of the menu and booking flows. Exact results depend on the records in the local MySQL database.

Console Output — Booking Menu



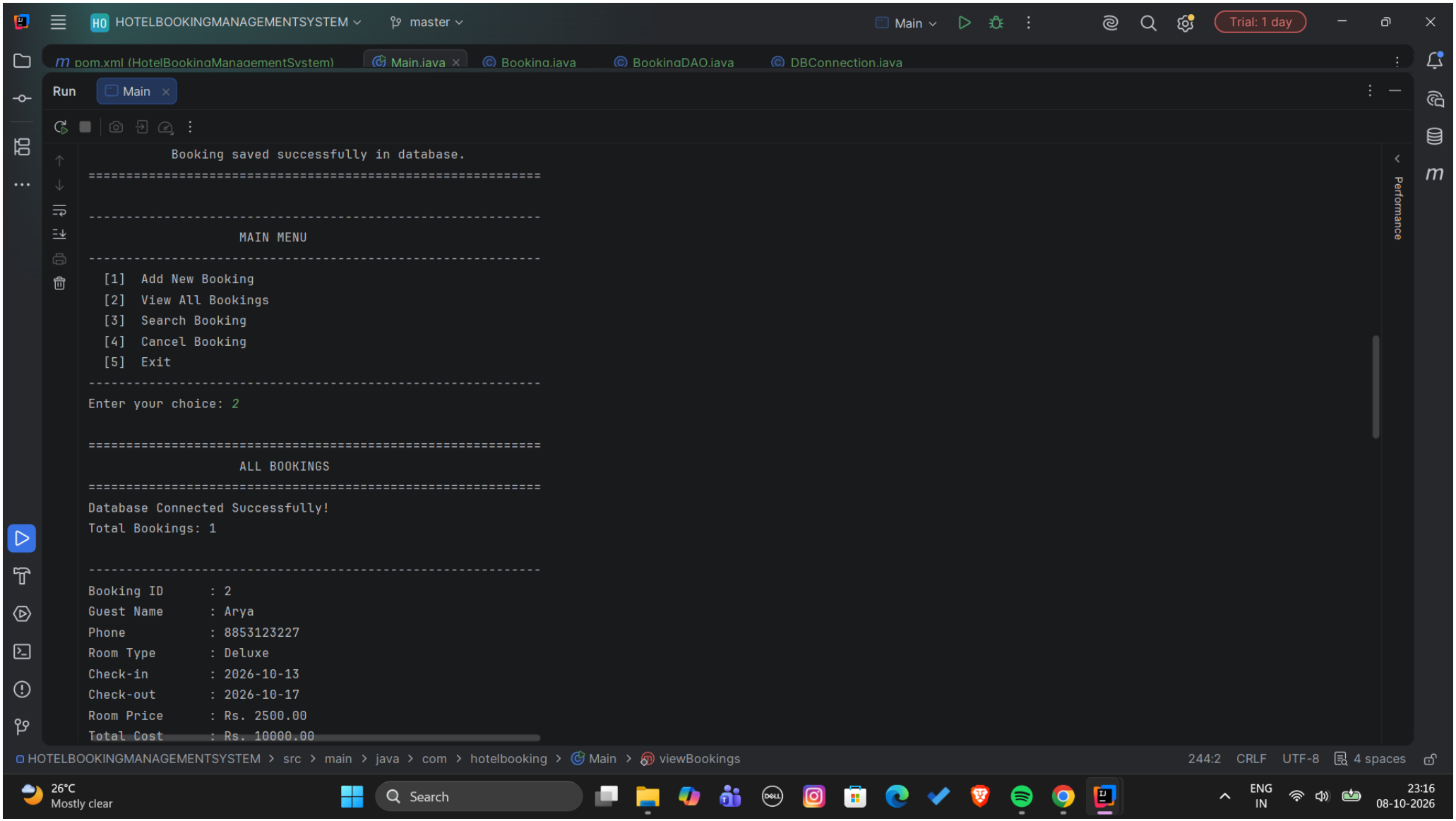
Submitted application output screenshot

Console Output — Booking Operations



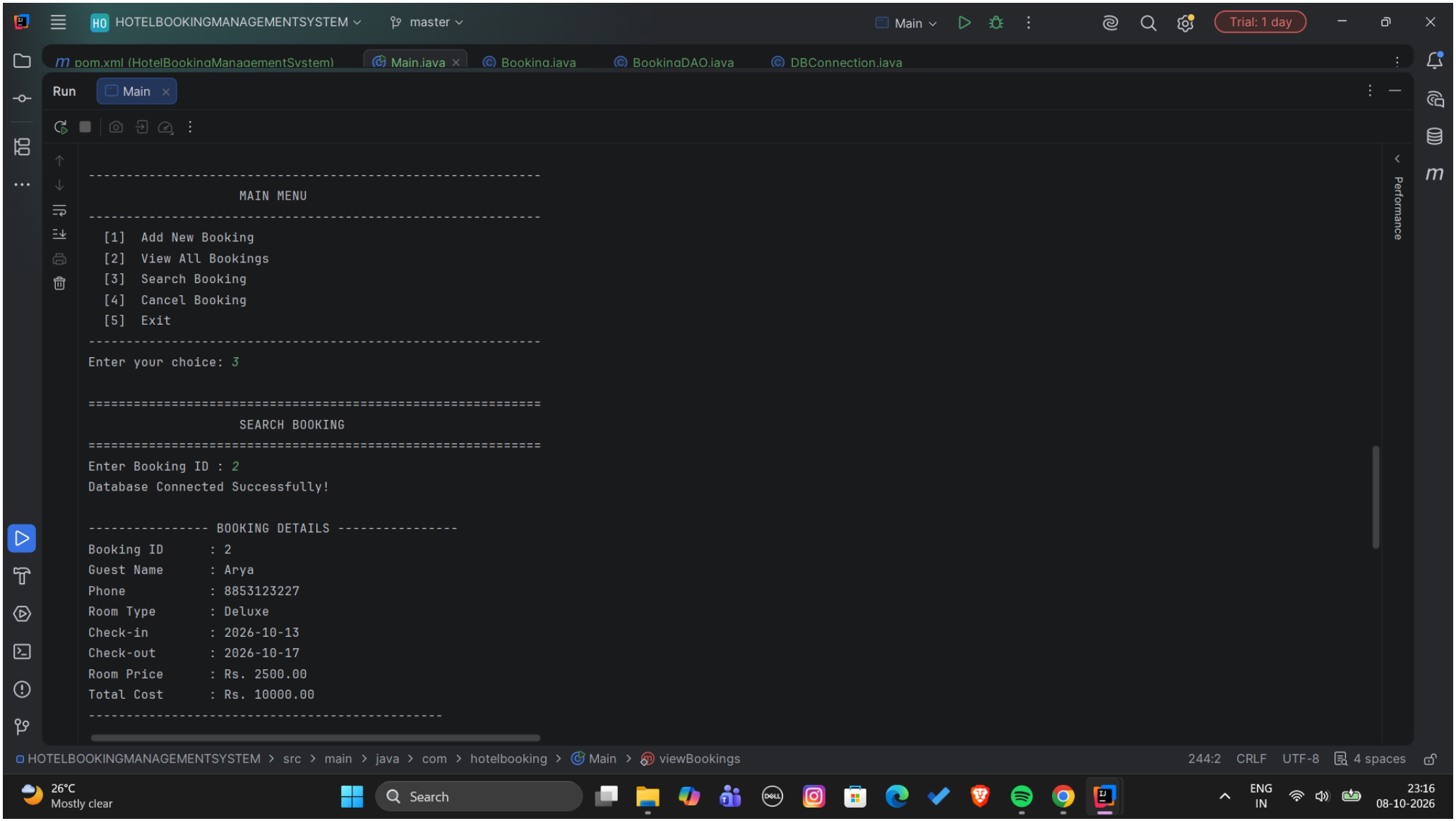
Submitted application output screenshot

Console Output — Booking Details



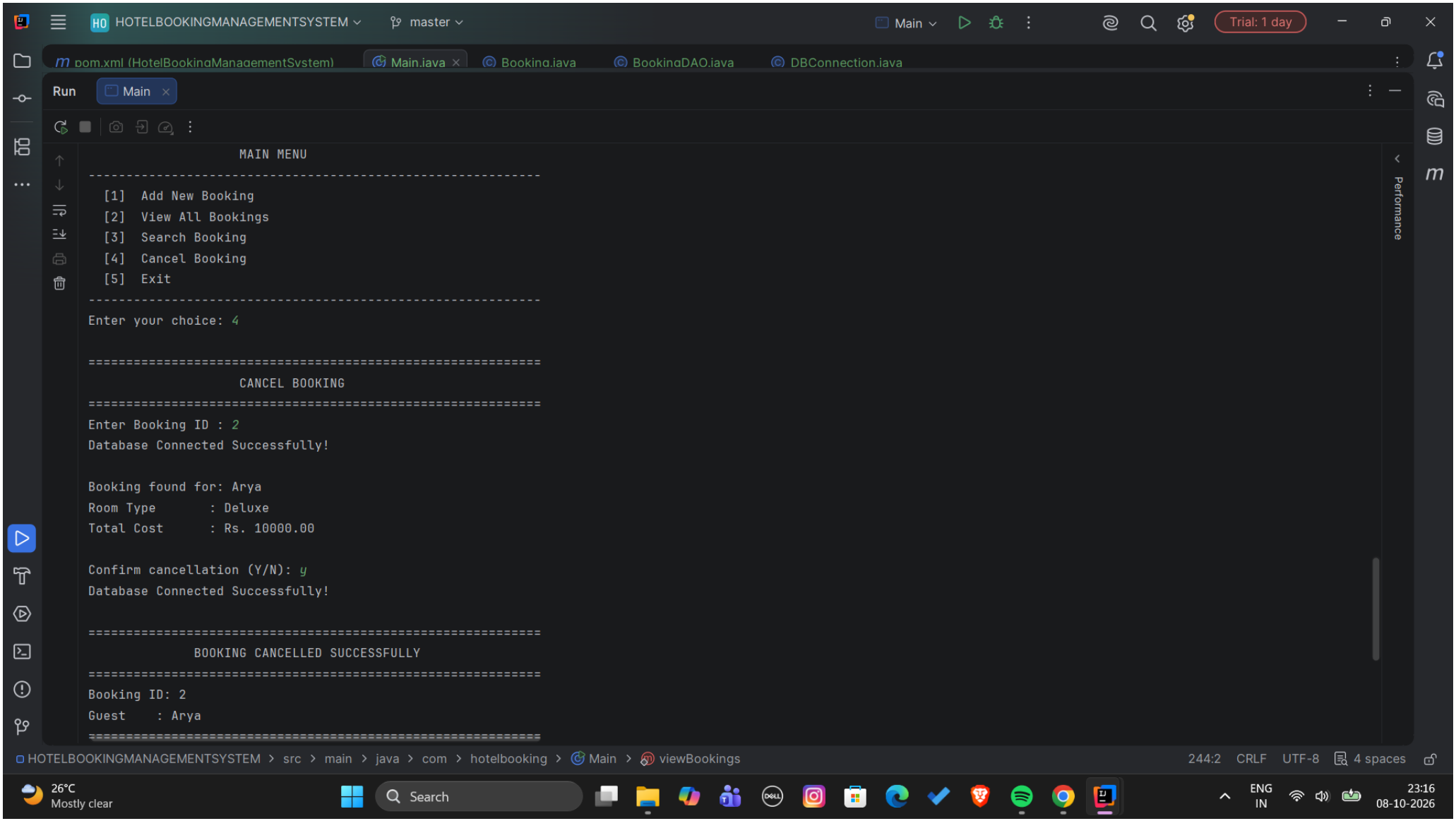
Submitted application output screenshot

Console Output — Database / Booking Results



Submitted application output screenshot

Console Output — Additional Test Run



Submitted application output screenshot

10. Testing Checklist

Test case	Expected result
Database connection	Application connects when URL, username and password are correct
Add booking	New booking is saved in MySQL
View bookings	Saved bookings are displayed
Search booking	Matching booking details are returned
Cancel booking	Selected booking is deleted and confirmation is shown
Invalid booking ID	No matching record is reported without crashing

11. Conclusion and Future Scope

This project demonstrates a layered Java application using Maven, JDBC and MySQL. The Booking model represents a reservation, BookingDAO performs database operations, and DBConnection centralizes connection settings. Future enhancements could include a graphical or web interface, room availability validation, date checks, authentication, billing, and configuration through environment variables.

Security: The database password has been obscured in the screenshot used in this report. Keep real credentials out of source control and rotate any password that was exposed publicly.